

Project - High Level Design

on

Real Estate Support Triage Agent

Institution Name: Medicaps University – Datagami Skill Based Course

Sr no	Student Name	Enrolment Number
01	Angel Gurjar	EN22CS301123
02	Kirti Choudhary	EN22CS301508
03	Mahi Vishwakarma	EN22CS301568
04	Pavani Kanoongo	EN22CS301699
05	Sahil Shrichand Talreja	EN22CS301838

Group Name: Group 01

Project Number: AAI-27

Industry Mentor Name:

University Mentor Name:

Academic Year:2022-2026

Table of Contents

Section No.	Section Title
1	Introduction
1.1	Scope of the Document
1.2	Intended Audience
1.3	System Overview
2	System Design
2.1	Application Design
2.2	Process Flow
2.3	Information Flow
2.4	Components Design
2.5	Key Design Considerations
2.6	API Catalogue
3	Data Design
3.1	Data Model
3.2	Data Access Mechanism
3.3	Data Retention Policies
3.4	Data Migration
4	Interfaces
5	State and Session Management
6	Caching
7	Non-Functional Requirements
7.1	Security Aspects
7.2	Performance Aspects
8	References

1. Introduction

The Real Estate Support Triage Agent is an AI-powered intelligent support automation system designed to streamline customer query management in real estate organizations. Real estate companies frequently receive high volumes of customer messages related to site visit scheduling, property documentation requests, payment confirmations, booking inquiries, and complaint reports. Traditionally, these messages are manually reviewed by support staff, who identify the intent, extract important details, and draft appropriate responses. This process is time-consuming, error-prone, and difficult to scale.

The proposed system leverages Large Language Models (LLMs), Retrieval Augmented Generation (RAG), and Agent-based tool orchestration to automate the triage process. The system classifies incoming messages based on intent, extracts structured entities such as flat ID, tower name, date, and booking ID, retrieves relevant contextual information from a vector database, and generates professional draft responses. The solution integrates FastAPI as the backend, Streamlit as the user interface, and Chroma DB as the vector database for knowledge retrieval.

This project demonstrates the application of modern AI techniques in building a production-style intelligent support system that enhances operational efficiency and improves response accuracy.

1.1 Scope of the Document

This document provides a comprehensive description of the Real Estate Support Triage Agent system, including its objectives, system architecture, design components, implementation methodology, and operational workflow.

The scope of this document includes:

- Problem definition and system motivation
- High-level and detailed architectural design
- Description of AI components such as Intent Classification, Named Entity Recognition (NER), and RAG
- Agent-based tool orchestration design
- Backend and frontend integration details
- Technology stack and development environment

- System workflow and interaction flow
- Testing scenarios and implementation roadmap
- Limitations and future enhancements

This document does not cover real-time deployment infrastructure or enterprise-level integration with live real estate management systems. The system is implemented as an academic prototype demonstrating applied AI engineering concepts.

1.2 Intended Audience

This document is intended for the following audience:

- Project Guide and Faculty Evaluators
- External Examiners during viva evaluation
- Academic Review Committees
- Students and researchers interested in applied AI system design
- Developers seeking to understand AI-based support automation architectures

The document assumes that the reader has a basic understanding of software engineering concepts, APIs, databases, and fundamental Artificial Intelligence principles.

2. System Design

The System Design of the Real Estate Support Triage Agent defines the architectural structure, major components, and interaction mechanisms used to implement the intelligent triage workflow. The system follows a modular and layered architecture to ensure scalability, maintainability, and clear separation of concerns.

The design integrates a user interface layer, backend API layer, AI processing layer, knowledge retrieval layer, and tool orchestration layer. Each component is responsible for a specific stage in the message processing pipeline, enabling efficient handling of real estate customer queries.

The architecture is designed to simulate a production-style AI support automation system while maintaining academic project feasibility.

2.1 Application Design

The application follows a multi-layered architecture consisting of the following primary components:

1. User Interface Layer (Streamlit)

- Provides a chatbot-style interface for interaction.
- Accepts customer messages as input.
- Displays: Classified intent , Extracted entities and Generated draft response
- Acts as the presentation layer of the system.

2. Backend API Layer (FastAPI)

- Exposes a REST endpoint (POST /chat).
- Receives user message from the UI.
- Forwards the message to the AI Agent.
- Returns structured output back to the UI.
- Ensures separation between frontend and AI logic.

3. AI Processing Layer (LLM Agent)

Implemented using CrewAI or LangChain.

Responsibilities:

- Perform Intent Classification
- Perform Named Entity Recognition (NER)
- Decide which backend tool to invoke
- Combine retrieved knowledge with response generation

This layer acts as the decision-making core of the system.

4. Knowledge Retrieval Layer (RAG + Chroma Vector DB)

- Stores property-related documents and data as embeddings.
- Performs semantic similarity search.
- Retrieves relevant context based on user query.

- Supplies contextual information to the LLM.

This improves factual grounding and reduces hallucination.

5. Tool Orchestration Layer

The agent dynamically invokes tools based on detected intent.

Defined tools include:

- `schedule_visit()` – Validates and confirms visit slots
- `fetch_documents()` – Retrieves requested property documents
- `generate_receipt()` – Generates payment receipt content
- `knowledge_search()` – Queries the vector database

This enables controlled execution logic rather than free-form LLM responses.

Architectural Interaction Summary

User → Streamlit UI → FastAPI → LLM Agent

→ Intent + NER → RAG Retrieval → Tool Invocation

→ Draft Response Generation → UI Display

This modular design ensures extensibility and maintainability.

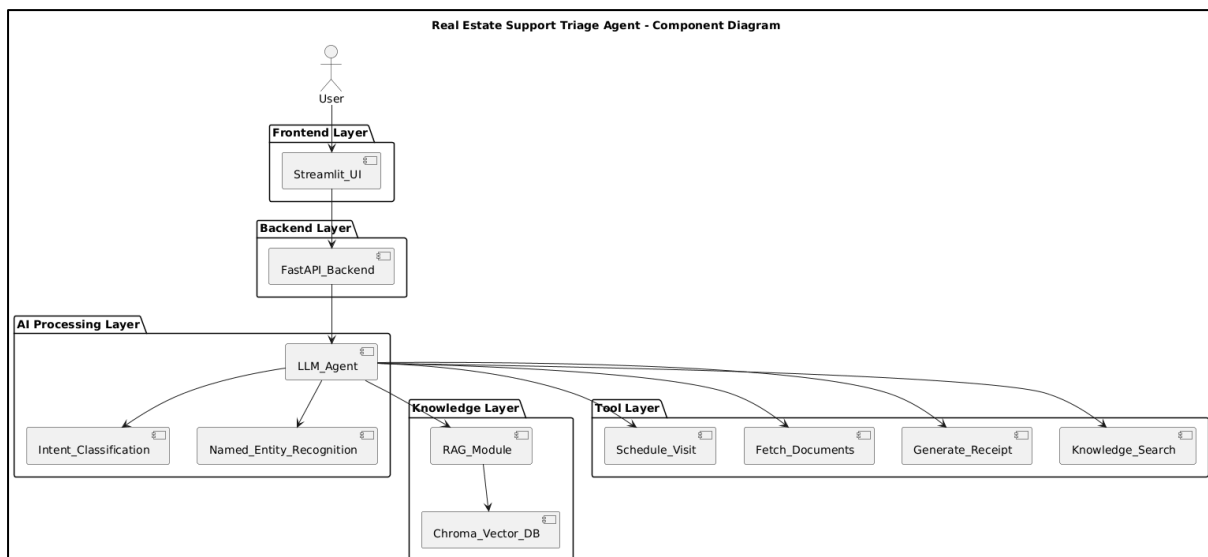


Fig 2.1 : Application Design

2.2 Process Flow

The process flow describes the step-by-step execution of the system from message input to response output.

Step 1: User Message Submission

- The user enters a query through the Streamlit chatbot interface.
 - Example:
"I want to schedule a visit for Flat A-102 tomorrow at 4 PM."

Step 2: API Request Handling

- The message is sent to the FastAPI backend via POST /chat.
- The backend forwards the message to the AI agent.

Step 3: Intent Classification

- The LLM analyzes the text.
- Determines the intent category:
 - Schedule Visit
 - Document Request
 - Payment Issue
 - Complaint
 - General Inquiry

Step 4: Named Entity Recognition (NER)

- Extracts structured entities:
 - Flat ID
 - Tower Name
 - Date
 - Time
 - Booking ID

These entities are converted into structured format.

Step 5: Knowledge Retrieval (RAG)

- The system performs semantic search in Chroma DB.
- Retrieves relevant property or document information.

- Passes retrieved context to the LLM.

Step 6: Tool Selection and Execution

- Based on classified intent, the agent selects appropriate tool.
- Example:
 - If intent = Schedule Visit → `schedule_visit()`
 - If intent = Document Request → `fetch_documents()`

Step 7: Draft Response Generation

- LLM combines:
 - Extracted entities
 - Retrieved knowledge
 - Tool output
- Generates a professional draft response.

Step 8: Response Delivery

- The backend returns:
 - Intent
 - Extracted entities
 - Generated draft reply
- Streamlit displays the output to the user.

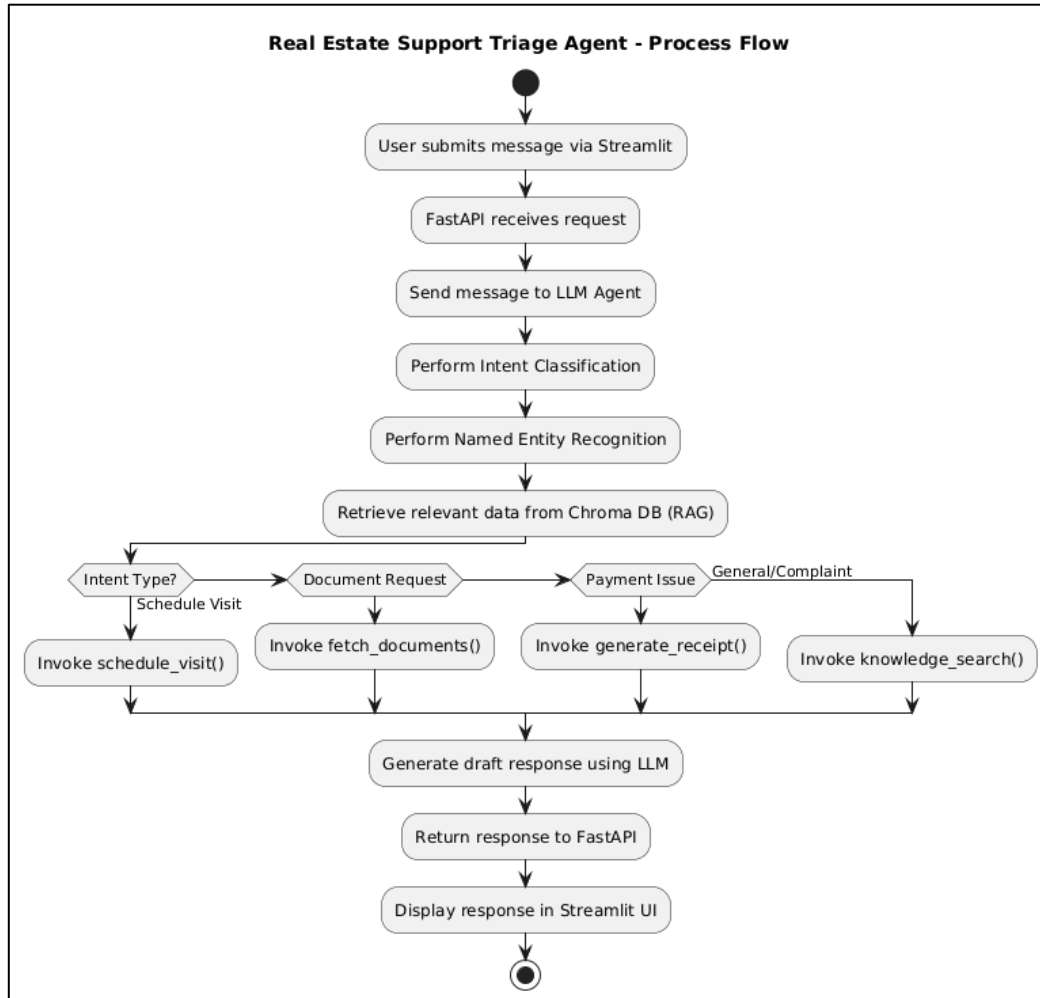


Fig 2.2: Process Flow

2.3 Information Flow

The Information Flow of the Real Estate Support Triage Agent explains how data moves across system layers and how user input is transformed into structured, context-aware output. The system follows a sequential processing pipeline where unstructured natural language text is converted into actionable information.

Input Stage

The process begins when a user submits a query through the Streamlit chatbot interface. The message is transmitted to the FastAPI backend via the /chat endpoint in JSON format. At this stage, the data is completely unstructured and represents natural language input from the user.

AI Processing Stage

Once received by the backend, the message is forwarded to the LLM-based agent. The agent performs multiple processing tasks:

- Intent Classification to determine the purpose of the query
- Named Entity Recognition (NER) to extract structured details such as flat ID, tower name, date, time, and booking ID

This stage converts unstructured text into partially structured data.

Knowledge Retrieval Stage

After classification and entity extraction, the system initiates the Retrieval Augmented Generation (RAG) process. The user query is converted into embeddings and compared against stored embeddings in the Chroma Vector Database. Relevant contextual information is retrieved and supplied to the agent to ensure domain-specific accuracy.

Tool Execution Stage

Based on the detected intent, the agent dynamically selects the appropriate backend tool. Examples include:

- `schedule_visit()` for visit booking
- `fetch_documents()` for document retrieval
- `generate_receipt()` for payment-related issues
- `knowledge_search()` for general queries

Each tool executes predefined business logic and returns structured output.

Response Generation Stage

Finally, the LLM integrates:

- Classified intent
- Extracted entities
- Retrieved contextual knowledge
- Tool output

Using this information, it generates a professional draft response. The backend returns the structured output to the Streamlit UI, where it is displayed to the user.

Overall, the information flow ensures controlled transformation of data from raw input to structured, intelligent output through a modular and sequential pipeline.

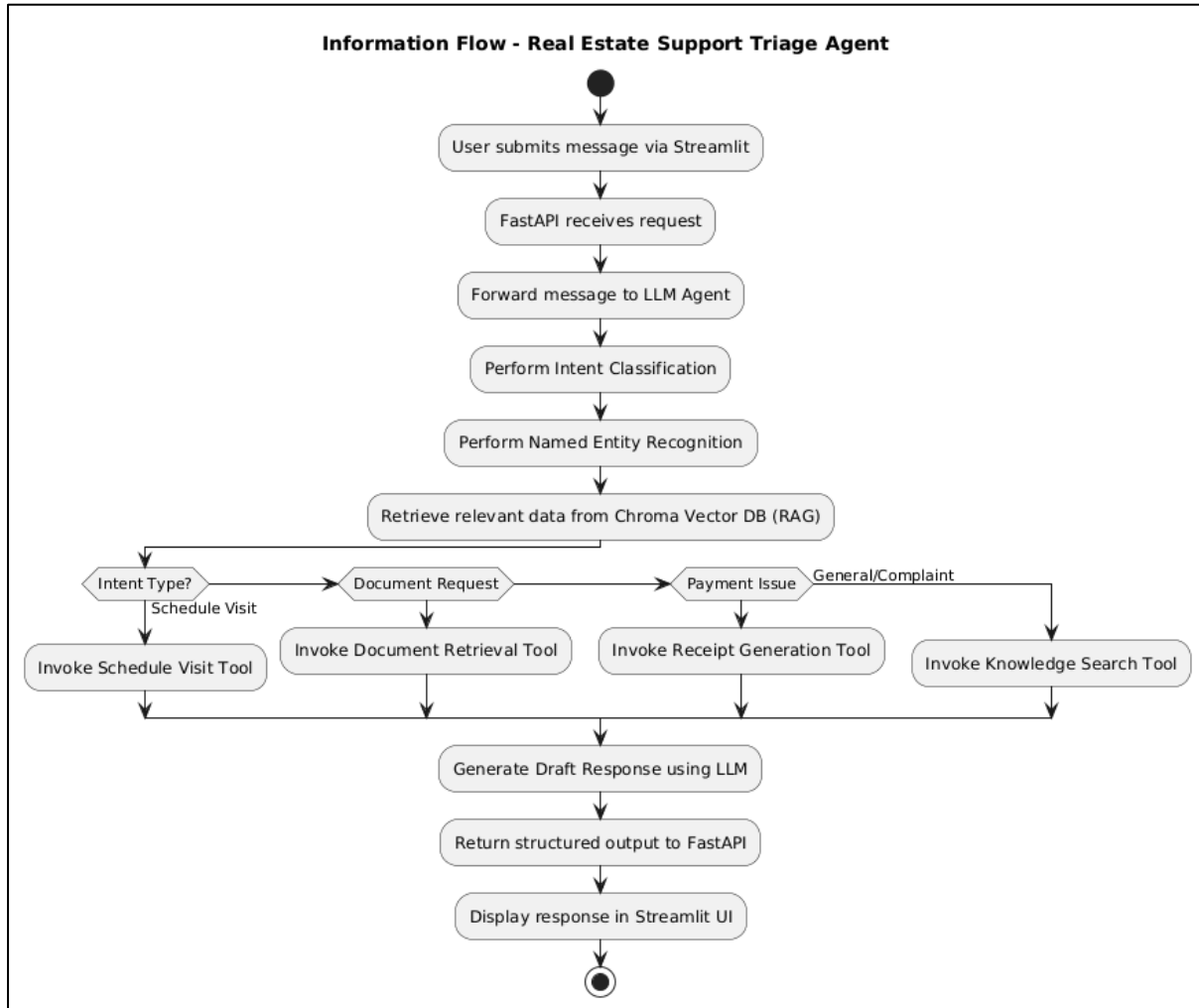


Fig 2.1 : Information Flow

2.4 Component Design

The Component Design describes the structural organization of the system and the responsibilities assigned to each module. The architecture follows a layered and modular design to ensure maintainability, scalability, and extensibility.

1. Streamlit UI Component

The Streamlit component acts as the presentation layer of the system. It is responsible for:

- Accepting user queries
- Displaying AI-generated responses
- Presenting extracted intent and entities

2. FastAPI Backend Component

The FastAPI backend functions as the intermediary between the UI and the AI processing layer. Its responsibilities include:

- Exposing the /chat endpoint
- Receiving and validating user input
- Forwarding requests to the LLM agent
- Returning structured responses

The backend is stateless and modular, allowing easy extension.

3. LLM Agent Component

The LLM Agent is the core intelligence module of the system. It performs:

- Intent classification
- Named Entity Recognition
- Tool selection
- Context-aware response generation

This component orchestrates interactions between the knowledge layer and tool layer.

4. RAG Module and Vector Database

The RAG module enhances factual accuracy by integrating domain-specific knowledge. Its responsibilities include:

- Generating embeddings from documents
- Storing embeddings in Chroma DB
- Performing semantic similarity search
- Retrieving relevant context for the LLM

This reduces hallucination and improves response precision.

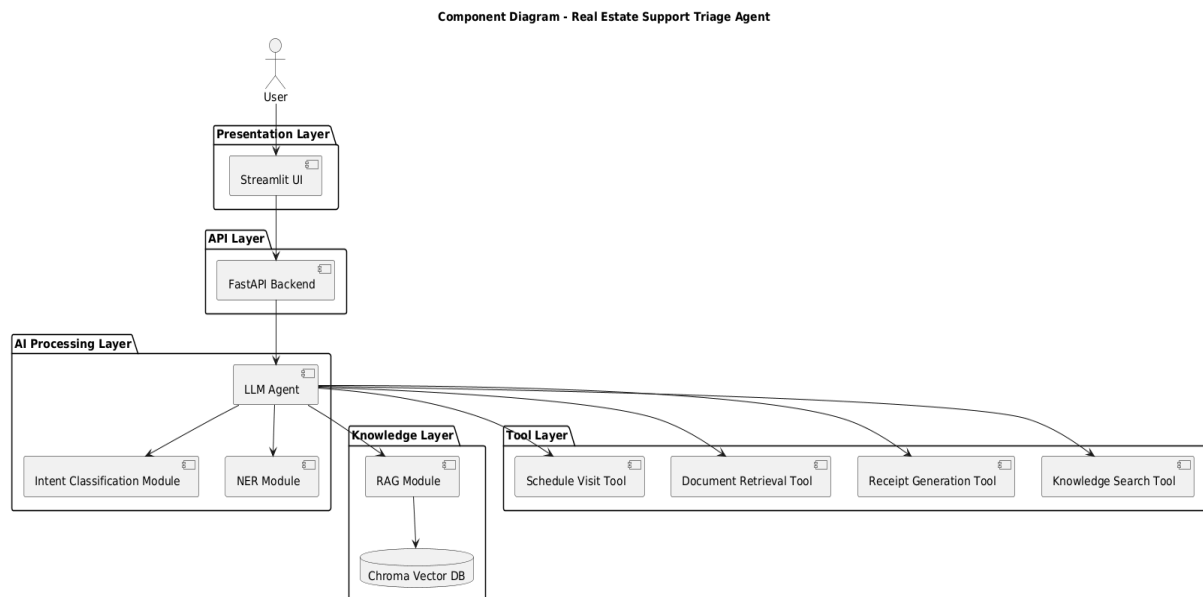
5. Tool Layer Components

The tool layer encapsulates business logic in independent modules such as:

- Visit scheduling

- Receipt generation
- Knowledge querying

Each tool operates independently and is invoked dynamically by the agent based on detected intent.



2.5 Key Design Considerations

The design of the Real Estate Support Triage Agent was guided by several technical and architectural considerations to ensure reliability, scalability, and maintainability. Since the system integrates modern AI components with traditional backend architecture, careful planning was required to balance flexibility and control.

Modularity

The system is designed using a layered and modular architecture. Each major functionality—UI, API layer, AI agent, RAG module, and tool layer—is implemented as an independent component. This separation ensures that updates or modifications in one

module do not affect other parts of the system. It also enables future extension, such as adding new tools or integrating additional AI models.

Scalability

The backend is implemented using FastAPI, which supports asynchronous request handling. The stateless design of the API ensures that the system can scale horizontally if deployed in a production environment. Additionally, the vector database (Chroma DB) is designed to handle large volumes of embeddings efficiently.

Accuracy and Reliability

To improve factual accuracy, the system incorporates Retrieval Augmented Generation (RAG). Instead of relying solely on pre-trained model knowledge, the LLM retrieves domain-specific data from the vector database before generating responses. This reduces hallucination and enhances response precision.

Controlled AI Execution

Rather than allowing unrestricted LLM responses, the system uses an agent-based tool orchestration approach. The agent selects predefined tools based on intent classification. This ensures that business logic execution remains deterministic and structured.

Maintainability

The use of clear module separation, consistent API interfaces, and encapsulated tool functions makes the system easier to maintain and debug. Future enhancements, such as database integration or authentication mechanisms, can be added without redesigning the entire architecture.

Extensibility

The architecture allows:

- Addition of new intent categories
- Integration of new tools
- Replacement of LLM providers
- Expansion of the knowledge base

This makes the system adaptable for real-world deployment scenarios.

2.6 API Catalogue

The API Catalogue defines the endpoints exposed by the backend and describes their functionality, request format, and response structure.

The backend is implemented using FastAPI and follows RESTful design principles.

2.6.1 Chat Endpoint

Endpoint

POST /chat

Description

Processes user messages, performs AI-based triage operations, and returns structured output including intent, extracted entities, and draft response.

Processing Steps

- Receive user message
- Forward message to LLM agent
- Perform intent classification
- Perform entity extraction
- Retrieve context using RAG
- Invoke appropriate tool
- Generate draft response

2.6.2 Internal Tool Interfaces (Logical APIs)

Although not exposed publicly, the agent interacts with internal tool modules. These include:

- `schedule_visit()` – Validates and confirms visit scheduling
- `fetch_documents()` – Retrieves property documentation
- `generate_receipt()` – Generates payment receipt details
- `knowledge_search()` – Queries vector database

These tools encapsulate business logic and return structured outputs to the agent.x

API Design Characteristics

- Stateless request handling
- JSON-based communication
- Modular endpoint structure
- Easily extendable for authentication or additional routes

The API Catalogue ensures clear documentation of system interfaces and supports maintainability and future integration with external systems.

3. Data Design

The Data Design of the Real Estate Support Triage Agent defines how information is structured, stored, processed, and retrieved within the system. Since the application integrates Large Language Models (LLMs) with Retrieval Augmented Generation (RAG), the data architecture includes both structured and unstructured data components.

The system primarily operates on unstructured textual input from users, which is transformed into structured representations during processing. Additionally, domain-specific property information is stored in a vector database to support semantic retrieval. The data design ensures consistency, efficient retrieval, and extensibility for future integration with persistent databases.

3.1 Data Model

The data model of the system consists of three primary data categories:

1. User Interaction Data
2. Extracted Structured Entities
3. Knowledge Base Data (Vector Embeddings)

1. User Interaction Data

User interaction data represents the incoming messages submitted through the chatbot interface. This data is initially unstructured natural language text. Each interaction is processed as an independent request.

Example structure:

- message: string
- timestamp: datetime (optional for logging)

This data is transient and processed in real time.

2. Extracted Structured Entities

During AI processing, the system converts unstructured text into structured entities using Named Entity Recognition (NER). These entities form a semi-structured data model.

Typical entity attributes include:

- intent: string
- flat_id: string
- tower_name: string
- date: string
- time: string
- booking_id: string

3. Knowledge Base Data (Vector Model)

The knowledge base stores property-related information, documentation details, and domain-specific text. Since the system uses Retrieval Augmented Generation, this data is transformed into vector embeddings before storage.

Each knowledge entry consists of:

- document_id
- raw_text
- embedding_vector

The embedding vectors are stored in Chroma Vector Database to enable semantic similarity search. This allows retrieval of contextually relevant information during user queries.

Data Model Characteristics

The overall data model supports:

- Transformation from unstructured to structured data
- Separation between operational data and knowledge base data
- Flexible schema for AI-based processing
- Compatibility with future relational database integration

3.2 Data Access Mechanism

The Data Access Mechanism defines how the system reads, processes, and retrieves data during execution. Since the project is implemented as an academic prototype, it does not rely on a traditional relational database for transactional storage. Instead, it uses a combination of real-time processing and vector database retrieval.

1. Real-Time Request Processing

User messages are transmitted from the Streamlit UI to the FastAPI backend using HTTP requests. The backend processes the data in memory and forwards it to the AI agent. No persistent storage is required for basic interaction flow.

Data flow mechanism:

- HTTP REST API communication
- JSON-based request and response format

2. Vector Database Access (Chroma DB)

For knowledge retrieval, the system uses Chroma DB as the vector database. The data access mechanism includes:

- Embedding generation using an embedding model
- Storage of embeddings in Chroma DB
- Semantic similarity search during query processing
- Retrieval of top relevant document chunks

The retrieval process involves converting the user query into a vector and performing similarity comparison with stored embeddings.

3. Tool-Level Data Access

The agent interacts with internal tools to execute business logic. These tools operate on structured entity data extracted during processing. Since the system is implemented as a prototype, tool logic may simulate operations such as slot validation or document retrieval.

Each tool:

- Accepts structured input
- Executes predefined logic
- Returns structured output

Data Access Characteristics

The system's data access mechanism is characterized by:

- Stateless API processing
- In-memory data transformation
- Semantic retrieval using vector similarity
- Modular access through agent-controlled tools

The design allows future extension to include persistent relational databases, authentication layers, or transaction logging mechanisms.

3.3 Data Retention Policies

Data Retention Policies define how long data is stored within the system and how it is managed over time. Since the Real Estate Support Triage Agent is implemented as an academic prototype, its data retention model is simplified. However, the design considers principles that would apply in a production deployment.

User Interaction Data

User messages processed through the Streamlit interface are handled in real time. By default, the system does not permanently store user conversations unless logging is explicitly implemented. This design ensures:

- Reduced storage overhead
- Improved privacy compliance
- Lower security risk exposure

In a production environment, user interactions may be stored for a defined retention period to support analytics, auditing, or performance evaluation. Such storage would typically follow organizational policies and regulatory compliance standards.

Extracted Structured Data

Entities such as flat IDs, dates, and booking IDs are generated during processing and used temporarily for tool execution and response generation. These structured entities are not persistently stored in the current implementation.

If integrated with a database in future versions, structured data retention would depend on:

- Business requirements

- Legal compliance
- Customer consent policies

Knowledge Base Data

The knowledge base, stored in the Chroma Vector Database, consists of property documents and embeddings. This data is retained as long as it remains relevant to system operations. Periodic updates may be performed to:

- Remove outdated property information
- Update pricing or documentation
- Refresh embeddings when models are upgraded

The retention of vector embeddings follows the lifecycle of the underlying knowledge documents.

Security and Privacy Considerations

Although the system operates as a prototype, data retention policies are designed to consider:

- Minimal data storage
- Controlled access to knowledge base
- Separation between user interaction data and knowledge data

In a real-world deployment, additional measures such as encryption, access control, and periodic data purging would be implemented.

3.4 Data Migration

Data Migration refers to the process of transferring data between storage systems, updating data formats, or upgrading system components without loss of integrity.

In the current implementation, large-scale migration is not required because the system primarily uses:

- Real-time API processing
- Chroma Vector Database for embeddings

However, future enhancements may require structured migration strategies.

Knowledge Base Migration

If the embedding model is upgraded or changed, existing vector embeddings may need regeneration. The migration process would involve:

1. Reprocessing raw documents
2. Generating new embeddings
3. Rebuilding the vector index in Chroma DB

This ensures compatibility between embeddings and retrieval mechanisms.

Database Integration Migration

If the system is extended to include a relational database (e.g., PostgreSQL or MySQL) for storing:

- User interaction logs
- Booking confirmations
- Transactional records

Then data migration may include:

- Schema transformation
- Data normalization
- Backup and restore operations
- Data validation after migration

Cloud Deployment Migration

In case of deployment to a cloud infrastructure, migration may involve:

- Moving local vector database to managed storage
- Containerization of application services
- Configuration updates for API endpoints

Migration Integrity Considerations

To maintain data integrity during migration, the following principles would be applied:

- Backup before migration
- Validation after migration
- Version control of data schema
- Testing in staging environment before production deployment

The data migration strategy ensures that the system remains adaptable to technological upgrades and infrastructure changes without compromising data consistency or system reliability.

4. Interfaces

The Interfaces section describes how different components of the Real Estate Support Triage Agent interact with each other and with external systems. The system follows a layered architecture, and communication between layers is achieved through well-defined interfaces. These interfaces ensure modularity, interoperability, and ease of future integration.

The system primarily includes three types of interfaces:

- User Interface
- Application Programming Interface (API)
- External Service Interfaces

4.1 User Interface

The User Interface is implemented using Streamlit and provides a chatbot-style interaction environment. This interface allows users to submit natural language queries related to real estate services such as visit scheduling, document requests, or payment issues.

The UI communicates with the backend through HTTP requests and displays the structured output returned by the system. It does not perform business logic or AI processing. Instead, it acts purely as a presentation layer responsible for:

- Accepting user input
- Displaying generated responses
- Presenting classified intent and extracted entities

The interface is designed to be simple, interactive, and suitable for demonstration purposes.

4.2 Backend API Interface

The backend interface is implemented using FastAPI and follows RESTful design principles. It serves as the communication bridge between the user interface and the AI processing layer.

The primary endpoint exposed by the system is:

POST /chat

This endpoint receives user messages in JSON format, forwards them to the AI agent for processing, and returns structured responses. The API ensures:

- Stateless communication
- Structured JSON request and response format
- Separation between frontend and processing logic

The API interface enables future integration with other frontend systems, mobile applications, or enterprise platforms.

4.3 AI Agent Interface

The AI Agent interface defines how the backend communicates with the LLM-based processing module. This interaction typically occurs internally within the application layer.

The agent receives raw text input and returns:

- Intent classification
- Extracted entities
- Tool invocation results
- Generated draft response

The agent interface encapsulates prompt engineering logic and tool orchestration mechanisms. This abstraction ensures that the LLM implementation can be modified without affecting other system components.

4.4 Vector Database Interface

The system interacts with the Chroma Vector Database for semantic retrieval operations. This interface supports:

- Storage of embeddings
- Similarity search operations
- Retrieval of relevant document chunks

The interaction between the RAG module and the vector database occurs through embedding generation and query-based similarity comparison. This interface enhances factual grounding of responses.

4.5 Tool Interface

The Tool Interface defines how the LLM agent interacts with internal functional modules such as visit scheduling and document retrieval. These tools are implemented as callable functions within the application.

Each tool:

- Accepts structured entity data as input
- Executes predefined business logic
- Returns structured output to the agent

This interface ensures controlled execution of business operations rather than unrestricted LLM-generated actions.

Interface Characteristics

The overall interface design of the system ensures:

- Clear separation of concerns
- Standardized communication protocols
- Modular integration capability
- Scalability for future extensions

By defining well-structured interfaces, the system maintains flexibility and adaptability for future enhancements such as authentication modules, database integration, or cloud deployment.

5. State and Session Management

The State and Session Management section describes how the Real Estate Support Triage Agent handles user interactions across requests and maintains contextual continuity during processing. Since the system follows a web-based client-server architecture, careful consideration is given to stateless communication and session control.

5.1 Stateless Backend Design

The FastAPI backend is designed to operate in a stateless manner. Each request sent to the /chat endpoint is treated as an independent transaction. The backend does not store user session data between requests by default. This approach offers several advantages:

- Improved scalability
- Simplified load balancing
- Reduced memory overhead
- Easier deployment in distributed environments

Because the API is stateless, any required contextual information must be included in the request payload.

5.2 Session Handling at UI Level

The Streamlit interface may optionally maintain session-level information during a user interaction. For example, it can temporarily store:

- Conversation history
- Previous responses
- Extracted entities

This session storage exists only during the active user session and is not permanently persisted unless logging is implemented. This allows the chatbot to simulate conversational continuity while keeping backend processing stateless.

5.3 AI Context Management

Although the backend remains stateless, the LLM Agent may utilize short-term conversational context within a single interaction cycle. Context management includes:

- Using previous messages (if passed explicitly)
- Incorporating retrieved knowledge from RAG
- Combining extracted entities with prompt templates

If multi-turn conversation is enabled in future versions, session identifiers could be introduced to maintain conversation history across multiple requests.

5.4 Vector Database State

The Chroma Vector Database maintains persistent state in the form of stored embeddings. Unlike session data, this knowledge base is long-term and remains available across system restarts. The state includes:

- Document embeddings
- Metadata associated with knowledge entries

This persistent state supports consistent retrieval behavior.

5.5 Future Enhancements for Session Management

In a production deployment scenario, session management may include:

- Unique session identifiers
- Authentication tokens
- Persistent conversation storage
- User-specific interaction history

Such mechanisms would enable personalized interactions and audit tracking while maintaining security compliance.

6. Caching

Caching refers to the temporary storage of frequently accessed data to improve system performance and reduce computational overhead. In the Real Estate Support Triage Agent, caching plays an important role in optimizing response time, especially in AI-driven and retrieval-based operations.

Although the current implementation functions as an academic prototype, the system design considers caching mechanisms that can enhance scalability and efficiency in production environments.

6.1 Purpose of Caching

The primary objectives of introducing caching in the system are:

- Reduce repeated computation for similar queries
- Minimize repeated embedding generation
- Improve response latency
- Reduce load on external LLM APIs
- Optimize vector database queries

Since LLM calls and vector similarity searches are computationally expensive, caching can significantly improve performance.

6.2 Caching in AI Response Generation

In scenarios where similar user queries are repeatedly submitted, the system may cache:

- Previously classified intents
- Extracted entity results
- Generated draft responses

By storing responses for identical or semantically similar queries, the system can return cached outputs instead of invoking the LLM again. This reduces API cost and improves responsiveness.

However, care must be taken to ensure that cached responses remain contextually valid and up-to-date.

6.3 Caching in RAG Retrieval

The Retrieval Augmented Generation (RAG) module performs embedding generation and similarity search operations. To optimize this process:

- Frequently accessed embeddings may be cached in memory
- Repeated semantic queries may reuse previous retrieval results
- Document chunks may be temporarily stored for faster access

This reduces repeated vector database lookups for common queries.

6.4 Application-Level Caching

Caching can be implemented at different layers of the system:

1. **UI-Level Caching**
Temporary storage of conversation state during a session.
2. **Backend-Level Caching**
In-memory caching of recent request-response pairs.
3. **Database-Level Caching**
Optimization within the vector database for similarity search.

In future production deployment, tools such as Redis or in-memory caching frameworks could be integrated for efficient cache management.

6.5 Cache Consistency Considerations

While caching improves performance, it must be managed carefully to prevent outdated responses. Key considerations include:

- Cache expiration policies

- Cache invalidation when knowledge base updates occur
- Ensuring updated embeddings replace older ones
- Avoiding stale business logic outputs

A well-defined invalidation strategy ensures system reliability.

7. Non-Functional Requirements

Non-Functional Requirements (NFRs) define the quality attributes of the Real Estate Support Triage Agent system. Unlike functional requirements, which describe what the system does, non-functional requirements describe how well the system performs its operations. These requirements ensure that the system is secure, reliable, scalable, maintainable, and efficient.

Given that the project integrates AI models, APIs, and a vector database, particular emphasis is placed on security and performance considerations.

7.1 Security Aspects

Security is a critical requirement, especially in systems that process user queries and potentially sensitive real estate information. Although the current implementation is an academic prototype, the architecture is designed with security best practices in mind.

Data Protection

User messages may contain sensitive information such as booking IDs or property details. Therefore:

- Data is processed in real time and not persistently stored by default.
- Structured entities extracted from user input are used only during processing.
- Minimal data retention reduces exposure risk.

In a production deployment, encryption mechanisms such as HTTPS would be used to secure communication between the UI and backend.

API Security

The FastAPI backend exposes a REST endpoint (POST /chat). To prevent misuse, the following security controls would be applied in a production environment:

- Authentication tokens (e.g., JWT-based authentication)
- Role-based access control
- Rate limiting to prevent abuse

- Input validation and sanitization

Input validation ensures protection against injection attacks or malformed requests.

LLM Interaction Security

Since the system interacts with external LLM APIs, secure handling of API keys is essential. API keys must be:

- Stored in environment variables
- Not hard-coded in source files
- Restricted through role-based permissions

Additionally, prompt engineering must ensure that malicious prompt injection attempts do not override system behavior or tool execution logic.

Vector Database Security

The Chroma Vector Database stores embeddings derived from domain-specific data. Security considerations include:

- Controlled access to embedding storage
- Prevention of unauthorized modification
- Backup mechanisms to prevent data loss

If deployed in production, the vector database would be secured using access control policies and possibly network isolation.

Future Security Enhancements

For enterprise-level deployment, additional security mechanisms may include:

- End-to-end encryption
- Audit logging
- Secure cloud deployment configurations
- Compliance with data protection regulations

7.2 Performance Aspects

Performance defines how efficiently the system processes user queries and generates responses. Since the system integrates AI-based processing and vector similarity search, performance optimization is an important consideration.

Response Time

The system is designed to generate responses within an acceptable time frame. The response time depends primarily on:

- LLM inference time
- Embedding generation time
- Vector similarity search latency
- Tool execution overhead

Optimizations such as caching and asynchronous API handling help reduce latency.

Scalability

The backend follows a stateless architecture, which allows horizontal scaling in a distributed deployment. If deployed on cloud infrastructure, multiple instances of the FastAPI service can handle concurrent requests.

The vector database is designed to handle increasing volumes of embeddings efficiently. As the knowledge base grows, indexing mechanisms ensure retrieval performance remains stable.

Concurrency Handling

FastAPI supports asynchronous request handling, allowing the system to process multiple requests concurrently. This improves throughput and ensures that the system can handle multiple users without significant degradation in performance.

In high-load scenarios, load balancing techniques can be implemented to distribute traffic across multiple service instances.

Computational Efficiency

LLM calls are computationally expensive. To maintain efficiency:

- Retrieval Augmented Generation reduces unnecessary token usage
- Tool-based orchestration avoids redundant LLM reasoning
- Caching mechanisms may reduce repeated computations

These optimizations reduce API cost and improve response stability.

Resource Utilization

The system is designed to operate within moderate computational resources suitable for academic demonstration. However, in production environments, resource planning would consider:

- CPU usage for backend services
- Memory requirements for embedding storage
- Network bandwidth for API communication

Efficient resource allocation ensures consistent performance.

8. References

- [1] FastAPI Documentation, "FastAPI – High Performance Web Framework for Building APIs with Python," Available: <https://fastapi.tiangolo.com>
- [2] Streamlit Documentation, "Streamlit – The Fastest Way to Build Data Apps," Available: <https://docs.streamlit.io>
- [3] LangChain Documentation, "LangChain – Building Applications with LLMs," Available: <https://python.langchain.com>
- [4] CrewAI Documentation, "CrewAI – Multi-Agent Orchestration Framework," Available: <https://docs.crewai.com>
- [5] Chroma Documentation, "Chroma – The AI-native Open-Source Embedding Database," Available: <https://docs.trychroma.com>
- [6] OpenAI, "GPT Models and API Documentation," Available: <https://platform.openai.com/docs>
- [7] Google AI, "Gemini API Documentation," Available: <https://ai.google.dev>
- [8] Anthropic, "Claude API Documentation," Available: <https://docs.anthropic.com>
- [9] J. Devlin et al., "BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding," Proceedings of NAACL-HLT, 2019.
- [10] P. Lewis et al., "Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks," Advances in Neural Information Processing Systems (NeurIPS), 2020.